NavSphere API reference

All URLs are relative to the configured backend. Production uses HTTPS. The final hardware requirement is **six** channels, GPIO32, GPIO33, GPIO34, GPIO35, GPIO36, GPIO39 in that order. No external acquisition hardware is used.

Authentication

• `POST /api/v1/auth/login` JSON `{ "username": "admin", "password": "admin" }` creates the development session. These credentials only seed a missing account in development. The password is hashed with scrypt.

• Login, `GET /api/v1/auth/me`, and `POST /api/v1/auth/password` return `{ "user": { "username": "admin", "role": "admin", "must_change_password": true }, "csrf_token": "..." }`.

• Send the HttpOnly `navsphere_session` cookie (`credentials: "include"` in fetch) and header `X-CSRF-Token` for every mutation after login.

• `POST /api/v1/auth/password`: `{ "current_password": "admin", "new_password": "a new password of at least 12 characters" }`. Password changes revoke other sessions. Commands are forbidden until the first password change.

• `POST /api/v1/auth/logout` revokes the session; no JSON body needed. Sessions expire after the configured TTL. At most ten sessions per account are retained.

• Login is limited to ten attempts per client IP per minute in the single backend process. Do not trust arbitrary proxy headers; terminate the service behind a correctly configured trusted reverse proxy in production.

Swagger `/docs` includes a header input for CSRF on each authenticated mutation. Execute login, copy the returned token, and enter it on password/logout/commands. The browser sends the session cookie automatically. `/redoc` and `/openapi.json` describe REST schemas; the MQTT schema is also available as `MqttTelemetry` in OpenAPI components.

Telemetry and device APIs

• `GET /api/v1/devices` returns `{ "devices": [{ "device_id", "online", "last_seen", "source", "boot_id", "controls": { "A": null, "B": null }, "mapping": [{ "channel": "analog_1", "label": "Analog 1", "physical": "GPIO32 / ADC1" }] }] }`. The mapping contains six entries. `controls` are nullable until a verified state is known; stale/offline devices show null controls.

• `GET /api/v1/devices/{device_id}/status` returns one device object. `online` requires recent telemetry and no explicit offline announcement.

• `GET /api/v1/telemetry/latest?device_id=ESP32_001` returns `{ "received": reading|null, "persisted": reading|null }`. Reading contains all MQTT fields below plus `id`, `received_at` and `persisted_at`. A received event has no database ID or persistence time. Persisted values are loaded from the database, including after a server restart.

• `GET /api/v1/telemetry/history?device_id=ESP32_001&start_ms=...&end_ms=...&limit=600` returns `{ "points": [{ "timestamp_ms": 1790000000000, "channels": { "analog_1": 1200 } }], "statistics": { "analog_1": { "min": 1100, "max": 1300, "avg": 1200, "count": 20 } }, "count": 20, "aggregated": false, "time_basis": "device_epoch_or_server_receive" }`. Each ordinary history point has all six channels. Invalid channel values are null and excluded from statistics. An unsynchronized sample uses server receive time on the history axis.

• `GET /api/v1/telemetry/channels/{channel_id}` has the same parameters/shape, restricted to one channel. IDs accept `1` through `6`, `analog-1` through `analog-6`, or `analog_1` through `analog_6`.

• History defaults to five minutes. The query window is at most 31 days and `limit` is 2-2000. Larger result sets are averaged in SQL time buckets; min/max/average statistics still cover all valid rows in the requested range. CSV exports made by the dashboard contain the returned points and may therefore be aggregated.

• `GET /api/v1/health` returns `status`, `mqtt_connected`, `database_connected`, `database_engine`, and `metrics`: `received`, `persisted`, `invalid`, `duplicates`, `dropped`, `database_errors`, `frequency_hz`, `queue_depth`, `websocket_dropped`, `last_received_at`, `last_persisted_at`. Counters cover the current backend process lifetime; frequency uses recent valid arrivals.

• `GET /api/v1/health/ready` performs a database check and returns HTTP 200 only if both the database and MQTT connection are ready; otherwise HTTP 503.

Commands

• Admin `POST /api/v1/commands`: `{ "device_id": "ESP32_001", "key": "A", "value": 1 }`. Returns HTTP 202 and a command object: `command_id`, `device_id`, `key`, `value`, `status`, `reason`, `boot_id`, `created_at`, `expires_at`, `updated_at`.

• Status is `pending`, `accepted`, `applied`, `rejected`, or `timed_out`. A successful MQTT publish remains pending. Only the matching, timely, boot-bound device acknowledgement permits applied status.

• `GET /api/v1/commands/{command_id}` returns the command object. `GET /api/v1/commands?device_id=ESP32_001&limit=30` returns `{ "commands": [...] }` newest first; maximum limit 100.

• Commands require the admin role, changed initial password, CSRF token, connected broker, and fresh device. All requests/results are audited. Terminal results cannot regress on duplicate or reordered acknowledgements.

• Command lifetime is configurable from1-30seconds (default10), matching the supplied firmware/simulator maximum30seconds. If firmware `CONFIG_NAV_COMMAND_MAX_TTL_MS` is reduced, reduce backend `COMMAND_TIMEOUT_SECONDS` accordingly. Output safety leases are separate from command delivery lifetime.

MQTT v1

Topics: `navsphere/{device_id}/telemetry`, `/command`, `/ack`, `/status`; prefix and allowed device IDs are configurable. Use QoS1. Command, acknowledgement and telemetry messages are not retained. Retained telemetry and acknowledgements are rejected. Broker ACLs and TLS bind each device to its own topics. Payload device IDs must match topic IDs.

```
{
  "schema_version": 1,
  "device_id": "ESP32_001",
  "boot_id": "a-new-id-each-boot",
  "sequence": 1,
  "timestamp_ms": 1790000000000,
  "time_valid": true,
  "uptime_ms": 100,
  "channels": {"analog_1": 1200, "analog_2": 1300, "analog_3": 1400, "analog_4": 1500, "analog_5":
  1600, "analog_6": 1700},
  "raw": [1489, 1613, 1737, 1861, 1985, 2109],
  "valid_mask": 63,
  "mqtt_connected": true,
  "dropped_samples": 0,
  "source": "simulator",
  "stale": false
}
```

Example values are illustrative. Values are millivolts, 0-5000; raw fields are unsigned16. Firmware may enforce tighter physical voltage limits. Exactly six values are required. Bit0 of `valid_mask` represents channel1; all valid is63. Sequence is unsigned32. Deduplication is unique `(device_id, boot_id, sequence)`, preserving sequence reuse after reboot. If time is unsynchronized, set `time_valid:false` and `timestamp_ms:null`. Synchronized timestamps must be year2000-2100 and at most five minutes into the future. `stale:true` preserves the original sequence and timestamp, so repeats cannot create new database measurements.

Commands published by the backend:

```
{"command_id":"550e8400-e29b-41d4-a716-
  446655440000","key":"A","value":1,"issued_at_ms":1790000000000,"expires_at_ms":1790000010000}
```

Acknowledgements publish accepted then applied, or rejected:

```
{"command_id":"550e8400-e29b-41d4-a716-
  446655440000","key":"A","value":1,"status":"applied","reason":"GPIO25 set high","boot_id":"a-new-
  id-each-boot"}
```

Status example (retained): `{"online":true,"boot_id":"a-new-id-each-boot","controls":{"A":0,"B":0}}`. Configure device Last Will as `{"online":false}`. An online announcement alone does not refresh stale telemetry. Report control states only after the corresponding physical state has been set. Firmware A maps to GPIO25 and B to GPIO26, active high; safe startup/disconnection level is0.

WebSocket

`/ws/v1/telemetry` authenticates using the same cookie and validates browser Origin. Maximum100 clients; each client has a32-event queue. Slow consumers lose oldest events, counted in metrics. Connections revalidate session revocation/expiry within15seconds.

- `{ "type": "telemetry", "stage": "received", "data": reading }`
- `{ "type": "telemetry", "stage": "persisted", "data": reading }`
- `{ "type": "command", "data": command }`
- `{ "type": "heartbeat", "health": health }` during idle periods.

Reconnect with bounded backoff, then retrieve latest/history to fill gaps. WebSocket delivery is a live view, not a durable event log.

Errors use FastAPI `{ "detail": "message" }`, or a validation error list. Typical statuses:401 unauthenticated,403 role/CSRF/password restriction,404 unknown device/channel/command,409 stale device,422 invalid input/range,429 login limit,503 dependency unavailable.